Errata and model clarifications — *Tamper-Proofing with Self-Modifying Code*

Errata to G. Morse and T. Kozsik, arXiv:2604.12407v1

Each item quotes the TeX source of arXiv:2604.12407v1 exactly (*before*), gives the replacement (*after*), a one-sentence reason and its basis. *Lean*: proved in the accompanying Lean development (declaration names given); *listing*: checked against assembled objects or native runs (evidence, not proof); *editorial*: an unambiguous internal inconsistency or a primary source. Page numbers refer to the arXiv v1 PDF. Items marked *open* await the authors' confirmation and are not applied.

Corrections

P01 (p. 8, Section 7, display after "back-of-the-envelope estimate"; Lean). Before: `\frac{10{,}000 \cdot (467{,}286 - 414{,}188)}{2{,}158{,}056} \approx 246, \qquad \frac{246}{10{,}000} \approx 2.5\%,`. After: `\frac{10{,}000 \cdot (467{,}286 - 414{,}188)}{2{,}158{,}056 - 414{,}188} \approx 304, \qquad \frac{304}{10{,}000} \approx 3.0\%,`. The estimate must divide by max - min: if an undisturbed run takes the minimum and a disturbed one the maximum, the disturbed fraction is (avg - min)/(max - min), not (avg - min)/max.

P01.2 (p. 8, Section 7, sentence after the display; Lean). Before: `gives roughly $3.6\%$.`. After: `gives roughly $7.8\%$ (Table~\ref{tbl:dynresult}, non-SMC columns, with the minimum subtracted in the same way).`. Same correction as P01 for the non-SMC columns: (42,300,840 - 39,600,010)/(74,295,022 - 39,600,010) = 7.78%, whereas 3.6% divides by the maximum.

P02 (p. 8, Section 7, paragraph introducing Table 5; Lean). Before: `about $2.5\times$ faster than the static SMC implementation`. After: `about $17.8\times$ faster than the basic static SMC implementation (Table~\ref{tbl:staticresult})`. Table 4 over Table 5 gives 8,305,389/467,286 = 17.8 (RDTSCP averages); no ratio of the tables' entries is 2.5.

P03 (p. 9, Caption of Table 4; editorial). Before: `the checksum covered the entire 220~KB code segment.`. After: `the checksum covered the entire 32.5~KiB (0x8200-byte) code segment.`. Figure 5 prints a code-segment size of 0x8200 bytes, and the clear count in Table 6 equals two clears per 8-byte word of exactly that region.

P04 (p. 2, Section 1, paragraph "Model"; Lean). Before: `returns true if and only if the protected code remains unmodified.`. After: `returns true for the unmodified protected code (when its timing lies within the calibrated bounds) and false for every modification that changes the checksum state or violates the time bound.`. A checksum is not injective, so "if and only if" is false; the provable statement is completeness for the genuine code and detection of every modification that changes the checksum state or violates the time bound. *Lean:* `forgery_value`, `two_not_enough`.

P04.2 (p. 10, Section 9, first paragraph; Lean). Before: `returns true if and only if the protected code has not been modified.`. After: `returns true for the unmodified protected code (when its timing lies within the calibrated bounds) and false for every modification that changes the checksum state or violates the time bound.`. Same as P04.

P05 (p. 10, Table 6, row labels; listing). Before: `SMC Loop & 4597855 \\ \hline Dyn SMC Loop & 4492871 \\ \hline`. After: `SMC Loop & 4492871 \\ \hline Dyn SMC Loop & 4597855 \\ \hline`. In the measurement program the label "SMC Loop" was printed for the dynamically generated kernel and "Dyn SMC Loop" for the statically assembled one, so the counts under the two labels belong to the other variant.

P05.2 (p. 8, Section 7, first paragraph; listing). Before: `stable for fine-grained timing.`. After: `stable for fine-grained timing. Each timed interval also contained one console write and one performance-counter driver call, which add the same kind of overhead to every variant.`. The measured intervals were not limited to the kernel; the reader needs to know what else they contained.

P06 (p. 7, Caption of Figure 3; Lean). Before: `The three arguments are the start address \lstinline|pData|, the protected length \lstinline|ulSize|, and the function start address \lstinline|pTPCompOffset|; \lstinline|sum| stores the checksum result.`. After: `The function arguments are the start address \lstinline|pData|, the protected length \lstinline|ulSize| (a multiple of 8 bytes) and the result address \lstinline|pResult|; \lstinline|pTPCompOffset| is the address of the body of the kernel \lstinline|TPComp| and is computed before the loop (not shown), \lstinline|sum| is stored to \lstinline|*pResult|, and \lstinline|op| is the persistent opcode state. The emulation is exact for an 8-byte-aligned region when \lstinline|op| equals the opcode state of the kernel at entry.`. The caption lists pTPCompOffset as an argument and omits pResult; the figure also relies on an alignment/length precondition and on the opcode state persisting between calls.

P07 (p. 6, Table 2, CMP row, LAT column; editorial). Before: `CMP & 0x3B & 7 & 0.2 / 0.25`. After: `CMP & 0x3B & 7 & 1`. A latency of 0.2-0.25 cycles is not a possible latency (the number is a reciprocal throughput); the measured register-to-flags latency of CMP is 1 cycle.

P08 (p. 2, Section 1, paragraph "Model"; Lean). Before: `constant-time algorithmic kernels`. After: `algorithmic kernels whose control flow and instruction count are independent of the data`. The kernels execute a data-independent instruction sequence, but individual instructions still differ in cost (Table 2), so "constant-time" overstates the property.

P09 (p. 9, Section 8, first paragraph; Lean). Before: `$c_{\text{non-SMC}}\,\mathcal{O}(n) \gg c_{\text{SMC}}\,\mathcal{O}(n)$`. After: `$c_{\text{non-SMC}}\,n \gg c_{\text{SMC}}\,n$`. An asymptotic class absorbs constants, so a comparison of the two constants cannot be written with O(n) on both sides.

P10 (p. 6, Table 3; editorial). Before: `SHL & Shift left (unsigned)`. After: `SHL & Shift left (logical, same as SAL)`. SHL is the logical shift; "unsigned" is not the architectural name.

P11 (p. 10, Section 10 (Security assumptions and threat model), whole section; Lean). Before: `\section{Security`

assumptions and threat model}\label{sec:assumptions} // The scope of the model is defined by four main assumptions. \begin{enumerate} \item A von Neumann architecture is available; pure Harvard architectures fall outside the intended deployment model. \item An attacker cannot discover and neutralize all embedded checksum functions. \item There does not exist a semantics-faithful non-SMC implementation that is as fast as, or faster than, the SMC version under the same timing constraints. \item Specialized hardware with direct memory access, such as certain FPGA-based platforms, would require a correspondingly larger-area defense strategy. \end{enumerate} // These assumptions intentionally separate what is being claimed from what is not. For example, self-reconfigurable hardware effectively realizes SMC in another substrate and is therefore compatible with the model rather than excluded by it. Similarly, the mechanism is not claimed to be equivalent to cryptographic attestation; it is a local runtime-integrity technique intended to raise attacker cost substantially on commodity systems.. After: see Appendix (full replacement of the section). The section states no adversary capabilities, goal or security statement; assumption 3 is an uncheckable universal negative, assumption 4 a remark, and assumptions the scheme needs (time integrity, persistence, response coverage, non-compensation) are missing. *Lean:* `security`, `witness_secure`, `faithful_needs_med`, `mediated_timefail`, `T3`, `T4`, `tsc_low_level`, `ext_clock_detects`, `injected_code_outside_goal`, `T3_full`.

P12 (p. 2, Section 1, paragraph "External inputs"; Lean). Before: `moreover, such channels do not protect code at the machine-architecture level against manual debugging, thread injection, or remote process-memory writes.`. After: `moreover, such channels do not protect code at the machine-architecture level against manual debugging, thread injection, or remote process-memory writes. The mechanism proposed here does not prevent these either: it detects persistent modification of the checksummed bytes and interference inside timed intervals, but not injected code that leaves the checksummed bytes intact, register tampering outside timed intervals, or a modification that is undone before the next covering check (Section~\ref{sec:assumptions}).`. The sentence suggests that the proposed scheme protects against debugging, thread injection and remote memory writes; it does not. *Lean:* `injected_code_outside_goal`, `noeffect_events`, `necessary_A6`.

P13 (p. 2, Section 1, paragraph "Practical stance"; Lean). Before: `An attacker could attempt to simulate execution under a synthetic time source and adapt the predicate accordingly, but this requires a deterministic, fully characterizable timing model. Modern commodity processors do not satisfy this assumption.`. After: `An attacker who controls the time source need not model the processor at all: replaying readings recorded from native runs makes any threshold predicate on those readings pass. Timing-based detection therefore requires a time source outside the adversary's control, for example \texttt{RDTSC}/\texttt{RDTSCP} read inside the protected code against a user-mode adversary, or an external source against a kernel-level adversary. An adversary who must run the protected code under emulation with an honest clock needs a deterministic, fully characterizable timing model, and modern commodity processors do not admit one.`. Replaying recorded timer readings defeats any timing predicate when the adversary controls the clock, so a synthetic time source needs no timing model. *Lean:* `necessary_A5`, `nec_A5_osApi`, `T4`, `ext_clock_detects`.

P14 (p. 5, Table 1, AMD64 column (counter global control MSR and SMC event); editorial, open). Before: `Perf. Counter Global Control & 0x38F & 0xC0000301.` After: `(no change: the cells are correct).` Withdrawn: all four AMD cells are confirmed by AMD's primary documents. MSR C000_0301 (PerfCntrGlobalCtl, 'Core Performance Global Control Register', Figure 13-12, present only where CPUID Fn8000_0022 reports it), C001_0000 (PerfEvtSel0) and C001_0004 (PerfCtr0) are in the AMD64 APM Vol. 2 rev 3.40 (January 2023), the revision the paper cites; PMCx021 'Pipeline Restart Due to Self-Modifying Code' is in the AMD Family 10h BKDG (31116 rev 3.62) and Family 15h Models 00h-0Fh BKDG (42301 rev 3.14).

Appendix: replacement text for P11

```
\section{Security assumptions and threat model}\label{sec:assumptions}

We state the adversary, its goal, the assumptions the scheme needs, and what is proved.  The
  statements are formalised and checked in Lean (Section~\ref{sec:summary}); platform behaviour
  enters only as explicit contract fields (timing bounds, which capabilities a privilege level has),
  not as proved facts about hardware.

\paragraph{Adversary levels.}
L0: patches the protected bytes before execution.
L1: user mode; may write to the protected process while it runs, single-step or breakpoint it, run
  it under emulation, and hook operating-system timer interfaces.
L2: kernel or hypervisor; additionally may give instruction fetch and data reads different views of
  memory and may trap or offset the time-stamp counter.
L4: additionally has physical writers such as DMA devices.
(L3 is reserved and has the capabilities of L2 in the model.)
The \emph{goal} is that a protected byte other than a legitimately varying SMC opcode byte is
  executed in modified form while no alarm is effective.
An alarm is effective when a checksum unit whose own response code is unmodified sees a wrong
  result, or $k$ consecutive checks of one unit exceed its time threshold $\tau$.

\paragraph{Assumptions.}
A1 (coherence, replaces ``von Neumann architecture''): the fetch and data views are one image.
A2 (coverage, replaces ``cannot neutralize all functions''): every protected byte lies in the region
  of some checksum unit, each unit's own code is protected, the coverage graph of the units (a unit
  covers another when its region contains the other's code) is strongly connected, and at least one
  unit is never modified.
A3 (timing gap, replaces the non-existence of an equally fast non-SMC implementation): the native
  check time is at most $\tau$, and $\tau$ is below the lower bounds on emulation, on a debug trap,
  and on mirroring an opcode write.
A4 (no physical writer): no device other than the protected thread writes the protected memory; a
  larger-area defense narrows the window between check and use but cannot close it.
A5 (time integrity): every timer reading equals the elapsed time.
A6 (persistence): a modification persists through the next $k$ scheduled checks of each unit that
  covers it.
A7 (response coverage): each unit's alarm response code is part of its own protected code.
A8 (non-compensation): a modification inside the region of a running unit changes the unit's result.
The checksum is deliberately not collision resistant, so A8 is an assumption about the design, and
  it cannot be dropped: two modified quadwords can restore the final sum and opcode state.

\paragraph{Results.}
(T1) Under A1--A8 and the platform contract, no adversary achieves the goal.
(T2) With separate fetch and data views, a run that follows the SMC kernel's opcode trajectory needs
  one mirroring event per opcode-changing iteration, each costing at least the mirroring bound of
  A3; a plain checksum without opcode state can be blind to split views.
(T3) Each of A1, A2 (coverage, unit code protected, strong connectivity, one unmodified unit), A3,
  A4, A5, A6, A7 and A8 is necessary: for each there is a system and an adversary that satisfies all
  the others and wins.
(T4) With the inline time-stamp counter, levels L0 and L1 satisfy A1, A4 and A5; an operating-system
  timer interface can be hooked at L1; L2 breaks A1 and A5; L4 breaks A4.
With a time source outside the adversary's control, of resolution $r$ and jitter $j$, slowdowns of
  at least $j+2r$ are detected by a threshold calibrated on native runs.

\paragraph{Scope.}
Not claimed: protection against code injected outside the checksummed bytes, against register
  tampering outside timed intervals, or against a modification that is undone before the next
  covering check; guarantees against L2 and above without an external clock; equivalence to
  cryptographic attestation.
The mechanism is a local runtime-integrity technique intended to raise attacker cost on commodity
  systems, and the proofs cover the model above, not the hardware.
```